

Course Outline: Organizing My Frontend

Title: Organizing My Frontend **Complexity:** Normal **Prerequisite:** Week 5, Day 15 — React Fundamentals (Props, Hooks, Rendering) **Target Audience:** Beginner developers who have built their first React/Next.js components and need to learn how to structure a real project **Total Lessons:** 18 **Format:** Mixed (39% hands-on = 7 lessons)

Course Description

You've written your first React components, wired up hooks, and made things render. But now the project is growing — and suddenly you can't find anything. Everything lives in the same folder, files are named randomly, and adding a new feature means scrolling through hundreds of lines in a single component. This course solves that problem.

Organizing your frontend is not about following rules for the sake of rules. It's about making your codebase readable to your future self and to any AI coding agent you collaborate with. A well-structured project means faster iteration, fewer mistakes, and prompts that actually produce what you intend — because the agent can understand your codebase at a glance.

This learnpack covers the concept of modularization from first principles, walks through the standard Next.js folder architecture, and gives you a repeatable decision framework for where every new file belongs. By the end, you'll be able to look at any messy project and reorganize it confidently.

Course Philosophy

This course emphasizes:

- Path of least resistance:** Start with the simplest split (views vs. components) and add structure only when the project demands it
- Attention to detail:** Good structure is mostly about naming things carefully and being consistent — skills that transfer to every project you'll ever work on
- Human-in-the-loop:** A well-organized codebase is also a better-documented codebase for your AI agent — context engineering starts with folder structure

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Concept of Modularization	3
02 - Modularizing React Components	4
03 - The Standard Next.js Folder Architecture	3
04 - Naming Conventions and File Contracts	3
05 - Assessment	2
06 - Code Challenge	1
Total	18

Section 00: Welcome

00.0 Why Folder Structure Matters More Than You Think

Learning Objectives:

- Explain why project structure becomes a bottleneck as a codebase grows
- Connect good folder organization to faster AI-assisted development
- Describe what this course will cover and how the lessons are sequenced

Content Outline:

- The moment a project stops being simple: when one file is no longer enough
- Structure as communication: your codebase talks to you, your teammates, and your AI agent
- What this course covers: modules, folder architecture, naming conventions, and a decision framework

Transitions: This lesson sets the stage. The next section introduces modularization as a concept before we apply it to React specifically.

Key Principles:

- A file you can't find is a file that doesn't exist
- The AI agent reads your folder structure before it reads your code — context starts at the root
- Good structure is a habit, not a one-time decision

Examples:

- A `components/` folder with 40 files and no subfolders — finding the right one takes longer than writing the component
 - A Next.js project where `app/page.tsx` contains 600 lines because nothing was ever extracted
-

Section 01: The Concept of Modularization

01.0 What Is a Module and Why Does It Exist?

Learning Objectives:

- Define what a module is in the context of frontend development
- Explain the single responsibility principle in plain language
- Identify what qualifies as a natural boundary for a module

Content Outline:

- Definition: a module is a self-contained unit of code with a clear, single purpose
- Modules in everyday software: a login form, a price calculator, a date formatter
- The single responsibility principle (SRP): one module does one thing well
- How JavaScript/TypeScript handles modules: `import` and `export`

Transitions: Now that we know what a module is in theory, the next lesson shows what happens when you ignore modularization entirely — so the motivation to apply it becomes concrete.

Key Principles:

- A module should be understandable without reading its dependencies
- If you need to read three files to understand one thing, that thing is not a module — it's a tangle
- Modules are named after what they do, not where they live

Examples:

- A `formatCurrency.ts` file that exports one function — small, focused, easy to test
 - A `utils.ts` file with 30 unrelated functions — technically a module, but not a useful one
 - React components as modules: `<ProductCard />` is self-contained, `<EverythingPage />` is not
-

01.1 The Cost of Not Modularizing: Spaghetti Projects

Learning Objectives:

- Recognize the specific failure modes that emerge from poor modularization
- Describe how spaghetti structure affects both developers and AI agents
- Identify warning signs in a codebase that indicate modularization is overdue

Content Outline:

- What a spaghetti project looks like: one giant component, everything in the root, random filenames
- The compounding cost: every new feature becomes harder than the last
- How poor structure breaks AI-assisted development: the agent can't infer context from chaos
- Warning signs to watch for: scroll fatigue, copy-paste repetition, fear of changing anything

Transitions: We've seen the problem. The next lesson moves to practice: you'll look at real code examples and identify where modularization opportunities exist.

Key Principles:

- Technical debt compounds — a messy structure today is a broken structure in two weeks
- If you're afraid to edit a file, that file is too large and too responsible
- The goal isn't perfection on day one; it's making the next change easier than the last

Examples:

- A single `App.tsx` file that contains the navbar, product list, cart, and checkout form — all in one place
 - A project where fixing a button color requires searching across five files because styles are scattered
 - An AI agent that generates the wrong component because the folder structure gives no signal about what already exists
-

01.2 Recognizing Modularization Opportunities in Existing Code

Learning Objectives:

- Scan a given component file and identify at least three extraction candidates
- Explain the reasoning behind each extraction decision
- Distinguish between code that should be extracted and code that belongs together

Content Outline:

- The extraction checklist: repeated code, code with a distinct name, code that could be reused elsewhere
- How to think about boundaries: UI logic vs. business logic vs. utility logic
- Reading code like a refactoring tool: looking for natural seams

Transitions: Now that you can spot modularization opportunities, the next section applies this thinking specifically to React components — views, components, and the split between them.

Key Principles:

- Extract when you name the thing before you extract it — if you can name it, it's a module
- Don't extract prematurely: duplication is better than the wrong abstraction
- Boundaries are found, not invented

Examples:

- A 200-line component where lines 80–120 are clearly a `<UserCard />` that appears three times
- A function inside a component that formats a date — it has nothing to do with rendering and belongs in `libs/formatDate.ts`
- A block of `useState` + fetch logic that is identical in two different components — a custom hook waiting to be born

Hands-On Exercise: You are given a single `Dashboard.tsx` file (~150 lines) that contains a page header, a stats summary bar, a list of recent activity items, and a logout button. Your task:

1. Read through the file and annotate (with comments) where you would draw module boundaries
2. List each extraction candidate with a proposed filename and a one-sentence description of its responsibility
3. Identify which extractions are components, which are utility functions, and which could become a custom hook
4. Do NOT refactor the file yet — just map the plan

Success Criteria:

- At least 3 extraction candidates identified with clear names
 - Each candidate correctly categorized as component, utility, or hook
 - Reasoning for each decision written in plain language
-

Section 02: Modularizing React Components

02.0 Views vs. Components: The Fundamental Split

Learning Objectives:

- Distinguish between a view (page-level component) and a reusable component
- Apply consistent naming to views and components based on their role
- Explain why this split is the first and most important structural decision in a React project

Content Outline:

- Views: components that represent a full page or route — `HomePage`, `ProductDetailPage`, `LoginPage`
- Components: reusable UI pieces that appear in one or many views — `<ProductCard />`, `<NavBar />`, `<Modal />`
- The naming convention: views get the `Page` suffix, components do not
- Where they live: `/app` (or `/pages`) for views, `/components` for everything else
- Why this split matters: it tells you immediately whether a file is a destination or a building block

Transitions: Now that you can tell views from components apart, the next lesson shows you exactly how to extract a component from a view — the mechanical process of doing it.

Key Principles:

- A view orchestrates; a component performs — they have different jobs
- Never put reusable logic inside a view that won't be shared — but name it as if it will be
- The `Page` suffix is a contract: this file mounts once per route and is never imported by another component

Examples:

- `app/products/page.tsx` is a view — it's the products page; `components/ProductCard.tsx` is a component — it renders one product and is used by the view
 - A `<CheckoutForm />` component used both on the cart page and in a modal — classic reusable component
 - A file called `stuff.tsx` in the root — violates both the view/component split and the naming convention
-

02.1 When One File Becomes Two: Extracting a Component

Learning Objectives:

- Execute a component extraction from a view file step by step
- Correctly wire props between the parent view and the extracted child component
- Verify the extracted component renders identically to the original inline code

Content Outline:

- The extraction process: identify → name → create file → move code → wire props → delete original block
- Props as the contract between parent and child
- The rule of thumb: if you need to pass more than 4–5 props, reconsider the boundary
- Verifying the refactor: same output, cleaner parent

Transitions: You can now extract a single component. The next lesson covers how to group multiple components into folders — because a flat `/components` directory runs out of room fast.

Key Principles:

- Extract in one direction: parent keeps the state, child receives what it needs via props
- The extracted file should have zero knowledge of the view it came from
- A successful extraction means the view file got shorter and nothing broke

Examples:

- Extracting a `<RecentActivityList />` from a dashboard page: the list logic moves to its own file, the page passes an `activities` array as a prop
- Extracting a `<StatsBar />` that displays four numeric KPIs — receives `stats` object as a single prop

Hands-On Exercise: Using the annotated `Dashboard.tsx` from the previous lesson, extract exactly one component — the stats summary bar.

1. Create `components/StatsBar.tsx`
2. Move the relevant JSX and any local variables it needs into the new file
3. Define the props interface using a TypeScript interface (`StatsBarProps`)
4. Import and use `<StatsBar />` in `Dashboard.tsx`, passing the required data as props
5. Confirm the page renders identically before and after the extraction

Success Criteria:

- `StatsBar.tsx` exports a default functional component with typed props
- `Dashboard.tsx` imports `<StatsBar />` and passes the correct data
- No logic from `Dashboard.tsx` was duplicated — it was moved
- The page output is visually unchanged

02.2 Grouping by Feature vs. Grouping by Type 📖

Learning Objectives:

- Compare grouping components by type (all cards together, all modals together) vs. by feature (everything related to checkout together)
- Apply the correct grouping strategy based on project size and team structure
- Explain when to switch from type-based to feature-based organization

Content Outline:

- Type-based grouping: `/components/cards/`, `/components/modals/`, `/components/forms/` — good for small projects
- Feature-based grouping: `/components/checkout/`, `/components/auth/`, `/components/dashboard/` — better as projects scale
- Hybrid approach: shared UI in `/components/ui/`, feature-specific in `/components/[feature]/`
- The practical rule: start with type-based, switch to feature-based when a type folder has more than 8–10 files

Transitions: The next lesson makes this concrete with the modal reorganization exercise — you'll restructure a real folder scenario hands-on.

Key Principles:

- There is no universally correct answer — the right structure is the one your team navigates fastest

- Consistency beats perfection: pick a convention and apply it everywhere
- Feature-based grouping scales better because features grow independently

Examples:

- A small portfolio site: `/components/NavBar.tsx` , `/components/ProjectCard.tsx` , `/components/Footer.tsx` — flat is fine
- A medium e-commerce app: `/components/product/ProductCard.tsx` , `/components/product/ProductGallery.tsx` , `/components/cart/CartItem.tsx`
- An over-engineered small project: 40 files nested 4 levels deep for a site with 3 pages

02.3 The Modal Reorganization Exercise

Learning Objectives:

- Apply folder grouping decisions to a concrete, realistic restructuring scenario
- Justify file placement decisions using the principles covered in this section
- Move files and update import paths without breaking the component tree

Content Outline:

- The scenario: a project where two modal components live in the root of `/components`
- The problem: both modals share a base style, and a third modal is needed — where does it go?
- The solution pattern: create a `/components/modals/` subfolder and apply consistent naming
- Updating imports after moving files: the one step developers always forget

Transitions: You've now modularized components and organized them into folders. The next section zooms out to the full Next.js project structure — beyond `/components` — and introduces the other folders every serious project needs.

Key Principles:

- Moving a file is not refactoring — it's housekeeping; do it before it's painful, not after
- Every time you create a new subfolder, update your mental model of the project map
- Import paths are a contract — break them intentionally and fix them immediately

Examples:

- Before: `components/ConfirmModal.tsx` , `components/DeleteModal.tsx` — two files with no clear relationship in the folder
- After: `components/modals/ConfirmModal.tsx` , `components/modals/DeleteModal.tsx` , `components/modals/index.ts` (barrel export)
- A new `CreateUserModal.tsx` drops naturally into `components/modals/` — the structure guided the decision

Hands-On Exercise: You are given a project with the following structure:

```
components/
  NavBar.tsx
  Footer.tsx
  ProductCard.tsx
  ConfirmModal.tsx
  DeleteModal.tsx
  LoginForm.tsx
  RegisterForm.tsx
  SearchBar.tsx
```

Your task:

1. Reorganize this into a logical subfolder structure using either type-based or hybrid grouping — justify your choice in a comment at the top of a `STRUCTURE.md` file
2. Create the new folder structure (you can represent this as a file tree in `STRUCTURE.md`)

3. Identify which files need a barrel export (`index.ts`) and write those barrel files
4. Update the import path for `<ConfirmModal />` in a sample `app/settings/page.tsx` to reflect its new location

Success Criteria:

- Components are grouped into at least 3 subfolders with clear naming
 - Grouping strategy is named and justified
 - At least one `index.ts` barrel export is written correctly
 - The updated import path in `page.tsx` is correct
-

Section 03: The Standard Next.js Folder Architecture

03.0 Beyond `/pages` and `/components` : The Full Folder Map

Learning Objectives:

- Name the standard additional folders used in Next.js projects beyond `/app` and `/components`
- Describe the responsibility of each folder: `libs`, `types`, `hooks`, `utils`, `context`, `services`
- Place a given piece of code in the correct folder based on its responsibility

Content Outline:

- The core folders every Next.js project eventually needs:
 - `/app` — routes and page-level views (Next.js App Router convention)
 - `/components` — reusable UI components
 - `/hooks` — custom React hooks (`useAuth`, `useCart`, `useDebounce`)
 - `/libs` — third-party library configurations and wrappers (`axios` instance, `supabase` client)
 - `/types` — shared TypeScript interfaces and type aliases
 - `/utils` — pure utility functions with no React dependency (`formatDate`, `slugify`, `clamp`)
 - `/services` — functions that communicate with APIs (separate from components)
- The distinction between `utils` and `libs` : `utils` are functions you wrote, `libs` are wrappers around tools you installed
- The distinction between `hooks` and `utils` : `hooks` use React APIs (`useState`, `useEffect`), `utils` do not

Transitions: Knowing the folders is one thing. The next lesson gives you a decision framework — a repeatable set of questions that tells you where any new file belongs.

Key Principles:

- If a function uses `useState` or `useEffect`, it belongs in `/hooks`, never in `/utils`
- If a file configures a third-party tool, it belongs in `/libs`, not scattered in the component that uses it
- `/types` is the source of truth for data shapes — import types from here, never redefine them in components

Examples:

- A function that calls `axios.get('/api/products')` — belongs in `/services/productService.ts`, not inside a component
 - A `useLocalStorage` hook — belongs in `/hooks/useLocalStorage.ts`
 - A `formatPrice(cents: number): string` function — belongs in `/utils/formatPrice.ts`
 - The configured `axios` instance with base URL and auth headers — belongs in `/libs/axiosClient.ts`
-

03.1 Where Does This File Belong? A Decision Framework

Learning Objectives:

- Apply a step-by-step decision process to determine the correct folder for any new file
- Distinguish edge cases where a file could reasonably go in two places and make a justified choice
- Build the habit of deciding folder placement before writing the first line of a new file

Content Outline:

- The decision flowchart: four questions that route any file to the right folder
 1. Does it render JSX? → `/components` (or `/app` if it's a page)
 2. Does it use React hooks? → `/hooks`
 3. Does it communicate with an external API or service? → `/services`
 4. Does it configure a third-party library? → `/libs`
 5. Is it a pure function with no dependencies? → `/utils`
 6. Is it a TypeScript type or interface? → `/types`
- Handling ambiguity: when a file could go in two places, pick the one that makes the import path most readable
- The colocation exception: if a component needs a helper that is used nowhere else, it can live next to the component

Transitions: You now have a decision framework. The next lesson shows you what happens when no framework is applied — the anti-pattern of the disorganized codebase.

Key Principles:

- Decide the folder before you open a new file — the decision shapes the file's design
- Readable import paths are a quality signal: `import { formatDate } from '@utils/formatDate'` is clear; `import { formatDate } from '../../components/Dashboard'` is not
- Colocation is valid for truly local helpers — but if you copy the function twice, it belongs in `/utils`

Examples:

- `useProductFilters.ts` : uses `useState` and `useEffect` → `/hooks/useProductFilters.ts` ✓
- `slugify.ts` : pure function, no imports, no React → `/utils/slugify.ts` ✓
- `Product` interface with `id`, `name`, `price` fields → `/types/product.ts` ✓
- A `fetchProducts()` function that calls the API → `/services/productService.ts` ✓

Hands-On Exercise: You are given a list of 10 code snippets (described below). For each one, write the folder it belongs in and the filename you would give it, following the naming conventions from this course.

1. A function `truncateText(text: string, maxLength: number): string` — no React, no imports
2. A hook that tracks window scroll position using `useState` and a `useEffect` event listener
3. A TypeScript interface `Order` with fields `id`, `userId`, `items`, `total`, `status`
4. A function that calls `POST /api/orders` with an order payload and returns the response
5. An Axios instance configured with `baseUrl` and a default `Authorization` header
6. A React component that renders a single order row in a table
7. A function `calculateDiscount(price: number, percent: number): number`
8. A hook that reads and writes a value to `localStorage`, exposing `[value, setValue]`
9. A TypeScript type `Status = 'pending' | 'shipped' | 'delivered' | 'cancelled'`
10. A function that initializes the Supabase client with the project URL and anon key

Success Criteria:

- All 10 items correctly placed with a valid folder path and filename
- No utility functions placed in `/hooks` and no hooks placed in `/utils`
- API-calling functions in `/services`, not in components or hooks
- Configurations in `/libs`

03.2 Anti-Pattern: The Disorganized Codebase

Learning Objectives:

- Identify the three most common structural anti-patterns in React/Next.js projects
- Explain why each anti-pattern occurs and what it costs over time
- Describe the correct approach for each anti-pattern

Content Outline:

- **Anti-pattern 1 — The Flat Root Dump:** every file lives in `/components` with no subfolders, regardless of type or feature
 - Why it happens: it works at first, so there's no pressure to change it
 - The cost: after 15–20 files, navigation becomes slower than writing the component again
 - The fix: introduce subfolders when any type group exceeds 6–8 files
- **Anti-pattern 2 — Business Logic in Components:** API calls, data transformations, and formatting functions written directly inside a component's render function
 - Why it happens: it's the path of least resistance when you're moving fast
 - The cost: the component is impossible to test, and the logic can't be reused
 - The fix: move logic to `/services`, `/utils`, or a custom hook in `/hooks`
- **Anti-pattern 3 — Type Redefinition:** the same TypeScript interface (`Product`, `User`, `Order`) defined in three different files because it was faster than looking up the shared one
 - Why it happens: it feels quicker in the moment; the compiler doesn't stop you
 - The cost: when the data shape changes, you update one definition and forget the other two
 - The fix: all shared types live in `/types`; components import from there

Transitions: With anti-patterns identified, the next section focuses on the last layer of project organization: naming conventions and barrel exports — the details that separate a readable project from a great one.

Key Principles:

- Anti-patterns are not failures of skill — they are failures of habit; the fix is a checklist, not talent
- The earlier you address structure, the cheaper it is — restructuring a 5-file project takes 10 minutes; restructuring a 50-file project takes a day
- Your AI agent inherits your anti-patterns: a disorganized project produces disorganized generated code

Examples:

- Anti-pattern 1: a `/components` folder with `Card.tsx`, `Modal.tsx`, `Form.tsx`, `Header.tsx`, `Footer.tsx`, `Sidebar.tsx`, `Button.tsx`, `Input.tsx`, `Dropdown.tsx`, `Table.tsx`, `Badge.tsx` — all flat, no grouping
 - Anti-pattern 2: a `ProductListPage.tsx` with a 40-line `useEffect` that fetches, transforms, filters, and formats product data inline — none of it reusable
 - Anti-pattern 3: `Product` defined in `ProductCard.tsx`, `ProductListPage.tsx`, and `cartSlice.ts` — three slightly different versions causing a type error that takes an hour to trace
-

Section 04: Naming Conventions and File Contracts

04.0 Consistent Naming: PascalCase, kebab-case, and Index Files

Learning Objectives:

- Apply the correct naming case to components, hooks, utilities, and folders
- Explain what an `index.ts` file does and when to use one
- Describe why naming consistency is a quality signal for both humans and AI agents

Content Outline:

- PascalCase for React components: `ProductCard.tsx`, `NavBar.tsx`, `ConfirmModal.tsx`
- camelCase for hooks and utilities: `useAuth.ts`, `formatDate.ts`, `slugify.ts`
- PascalCase for TypeScript types and interfaces: `Product`, `UserProfile`, `OrderStatus`
- kebab-case for folders: `/components/product-card/`, `/hooks/use-auth/` — or PascalCase for component folders (either, but pick one and stick to it)
- The `index.ts` convention: a file that re-exports everything from a folder so the import path collapses from `import X from '@components/modals/ConfirmModal'` to `import X from '@components/modals'`
- When to add an `index.ts`: when a folder has 2+ files that are regularly imported together

Transitions: The next lesson puts `index.ts` files into practice — you'll write barrel exports and see how they clean up import paths across a project.

Key Principles:

- Naming conventions are team contracts — the moment you break one, the codebase has two conventions
- A file named after what it does is documentation: `useProductSearch.ts` tells you everything before you open it
- `index.ts` barrel files are optional — use them when they reduce noise, not as a reflex

Examples:

-  `components/ProductCard.tsx` → PascalCase, matches the component name inside
-  `hooks/useDebounce.ts` → camelCase starting with `use`, matches the hook convention
-  `components/product_card.tsx` → snake_case in a TypeScript project creates visual inconsistency
-  `hooks/Debounce.ts` → PascalCase for a hook breaks the `use` convention and makes it look like a component

04.1 Barrel Exports and Clean Import Paths

Learning Objectives:

- Write a correct `index.ts` barrel export file for a component folder
- Explain the trade-off between barrel exports and direct imports
- Update a component's import statement to use a barrel export path

Content Outline:

- What a barrel export is: a single `index.ts` that re-exports everything from a folder
- Syntax: `export { default as ProductCard } from './ProductCard'`
- The path alias: configuring `@/` in `tsconfig.json` so imports start from the project root
- The trade-off: barrel files simplify imports but can make tree-shaking less effective in large projects — at the beginner level, the readability gain outweighs the trade-off

Transitions: The next lesson is the final anti-pattern lesson for this course — naming chaos and import spaghetti — which shows exactly what a project without these conventions looks like.

Key Principles:

- A barrel file is a public interface for a folder — it controls what the outside world can see
- Path aliases (`@/hooks/useAuth`) are more readable and more resilient to folder moves than relative paths (`../../hooks/useAuth`)
- Don't barrel everything — only folders that are imported by multiple parts of the app

Examples:

- A `components/modals/index.ts` that exports `ConfirmModal`, `DeleteModal`, and `CreateUserModal` — any page can now do `import { ConfirmModal } from '@components/modals'`
- A `types/index.ts` that re-exports all shared interfaces — one import to get `Product`, `User`, and `Order`
- A `hooks/index.ts` — usually not worth it since hooks are imported individually

Hands-On Exercise: You are given the following folder structure:

```
components/  
  modals/  
    ConfirmModal.tsx  
    DeleteModal.tsx  
    CreateUserModal.tsx  
  forms/  
    LoginForm.tsx  
    RegisterForm.tsx
```

```
types/  
  product.ts  
  user.ts  
  order.ts
```

Your task:

1. Write a `components/modals/index.ts` that barrel-exports all three modal components
2. Write a `components/forms/index.ts` that barrel-exports both form components
3. Write a `types/index.ts` that barrel-exports all three type files
4. Rewrite the following import statements to use the new barrel paths (assume `@/` is configured):
 - `import ConfirmModal from '../../components/modals/ConfirmModal'`
 - `import { LoginForm } from '../../components/forms/LoginForm'`
 - `import { Product } from '../../types/product'`

Success Criteria:

- All three `index.ts` files use the correct re-export syntax
- All three import rewrites use the `@/` alias and the barrel path
- No direct file-level imports remain for the reorganized folders

04.2 Anti-Pattern: Naming Chaos and Import Spaghetti

Learning Objectives:

- Identify naming inconsistencies that signal a poorly maintained codebase
- Explain why relative import paths (`../../..`) are a maintenance liability
- Describe the specific fixes for each naming and import anti-pattern

Content Outline:

- **Anti-pattern 1 — Mixed naming conventions:** `ProductCard.tsx` next to `product-card.tsx` next to `productcard.tsx` — three files, three conventions, one folder
 - The fix: establish one convention in a `CONVENTIONS.md` or in your AI agent's rule files, and enforce it from day one
- **Anti-pattern 2 — Meaningless filenames:** `helpers.ts`, `utils2.ts`, `stuff.tsx`, `misc.ts` — files named after what they aren't instead of what they do
 - The fix: every file is named after its primary export; if you can't name it, split it
- **Anti-pattern 3 — Deep relative imports:** `import { useAuth } from '../../../hooks/useAuth'` — a path that breaks every time a file moves
 - The fix: configure `@/` path alias in `tsconfig.json` and use it everywhere
- **Anti-pattern 4 — Circular imports:** component A imports from component B, which imports from component A
 - Why it happens: logic that belongs in `/utils` or `/hooks` gets left in components, creating dependency loops
 - The fix: any shared logic moves to a neutral folder that nothing else imports from

Transitions: This is the last conceptual lesson. The next section moves to assessment and then the code challenge, where you'll apply everything from this course to a realistic project scenario.

Key Principles:

- A filename is a promise — if the file doesn't keep that promise, rename it or split it
- Relative paths are a code smell when they climb more than one level: `../Component` is fine, `../../../../utils` is not
- Circular imports are a sign that your module boundaries are wrong, not that your import syntax is wrong

Examples:

-  A project with `Button.tsx`, `button.tsx`, `ButtonComponent.tsx`, and `btn.tsx` in the same folder — four names for the same concept

- ❌ `import { something } from '../../../../../../../shared/utils/helpers'` — this path will break the next time anyone reorganizes a folder
 - ✅ `import { something } from '@/utils/helpers'` — immune to folder restructuring
 - ❌ `Header.tsx` imports `ThemeContext` from `Footer.tsx` — Footer shouldn't own a context; move it to `/context/ThemeContext.ts`
-

Section 05: Assessment

05.0 Refactor a Messy Project Structure

Learning Objectives:

- Apply the complete set of modularization and naming decisions from this course to a realistic project
- Produce a reorganized folder structure with justified decisions for every change
- Update import paths to reflect the new structure

Content Outline:

- Review of the decision framework from Section 03
- Applying the view/component split, folder grouping, naming conventions, and barrel exports together
- Documenting restructuring decisions for a teammate (or AI agent) to understand

Transitions: This exercise is the practical capstone before the knowledge check. It tests whether you can apply the course concepts to a messy real-world scenario.

Key Principles:

- Refactoring structure is not rewriting code — the logic stays the same, the organization improves
- Document your decisions as you make them — the reasoning is as valuable as the result
- A good restructure leaves the codebase easier to add to, not just cleaner to look at

Examples:

- Moving `Dashboard.tsx` (600 lines) to `/app/dashboard/page.tsx` and extracting `/components/dashboard/StatsBar.tsx`, `/components/dashboard/ActivityFeed.tsx`
- Renaming `utils2.ts` to `formatters.ts` after identifying that all its functions format display values

Hands-On Exercise: You are given the following project with messy structure:

```
src/
App.tsx           ← 400 lines: contains routing, layout, and three full page views
helpers.ts        ← formatDate, slugify, calculateDiscount, fetchProducts, useScrollPosition
types.ts          ← Product, User, Order, CartItem, Status (all in one file)
Modal.tsx         ← a generic modal base component
ConfirmDeleteModal.tsx ← uses Modal.tsx
ProductCard.tsx   ← uses Product type
product-list.tsx  ← a page that lists products; uses ProductCard and fetchProducts
LoginPage.tsx     ← a login page with an inline fetch to POST /api/auth/login
login-form.tsx    ← a form component used by LoginPage
useAuth.ts        ← a hook that reads/writes a token in localStorage
```

Your task:

1. Design the target folder structure (write it as a file tree)
2. For every file, state where it moves and why — one sentence per file
3. Identify which items in `helpers.ts` belong in `/utils`, which in `/services`, and which in `/hooks`, and write the target filenames

4. Split `types.ts` into appropriately named files under `/types`
5. Identify one `index.ts` barrel export worth creating and write it

Success Criteria:

- All files placed in correct folders with valid justifications
 - `helpers.ts` decomposed into at least 3 separate files in the right folders
 - `types.ts` split into individual type files
 - One barrel `index.ts` written with correct re-export syntax
 - No business logic left inside page-level components after the refactor plan
-

05.1 Knowledge Check: Frontend Modularization

Learning Objectives:

- Demonstrate understanding of module boundaries, folder architecture, and naming conventions
- Apply the decision framework to classify files by type and location
- Identify anti-patterns and select the correct remediation

Question Format: Multiple choice (6 questions)

Questions:

Q1. A developer writes a function `slugify(text: string): string` that converts "Hello World" to "hello-world". The function uses no React APIs and no third-party imports. Where should this file live?

- A) `/hooks/slugify.ts`
 - B) `/utils/slugify.ts` 
 - C) `/libs/slugify.ts`
 - D) `/services/slugify.ts`
-

Q2. A project has this import: `import { ProductCard } from '../../components/product/ProductCard'`. The team has configured `@/` as a path alias pointing to `/src`. Which of the following is the correct refactored import?

- A) `import { ProductCard } from 'components/product/ProductCard'`
 - B) `import { ProductCard } from '@components/product/ProductCard'` 
 - C) `import { ProductCard } from '@ProductCard'`
 - D) `import { ProductCard } from '/src/components/product/ProductCard'`
-

Q3. A developer needs to create a function that calls `GET /api/products` and returns the list. Where should this function live?

- A) Inside the `ProductListPage.tsx` component, in a `useEffect`
 - B) In `/utils/productUtils.ts`
 - C) In `/services/productService.ts` 
 - D) In `/hooks/useProducts.ts`
-

Q4. A project has 12 components in a flat `/components` folder. A new modal component needs to be added. What is the best structural decision at this point?

- A) Add the new modal to the flat `/components` folder — consistency matters more than subfolders
 - B) Create a `/components/modals/` subfolder, move existing modal components there, and add the new one 
 - C) Create a new `/modals/` folder at the project root, separate from `/components`
 - D) Add the modal directly to the `/app` folder next to the page that uses it
-

Q5. A developer defines the `User` TypeScript interface inside `UserCard.tsx` because the component uses it and it felt faster. Two weeks later, `LoginPage.tsx` also needs the `User` type and redefines it slightly differently. Which anti-pattern does this

describe?

- A) The Flat Root Dump
- B) Import Spaghetti
- C) Type Redefinition ✓
- D) The God Component

Q6. A `components/modals/index.ts` file contains the following line:

```
export { default as ConfirmModal } from './ConfirmModal';
```

What is the purpose of this file?

- A) It prevents `ConfirmModal` from being imported by other components
- B) It creates a barrel export so `ConfirmModal` can be imported as `import { ConfirmModal } from '@components/modals'` ✓
- C) It replaces the `ConfirmModal.tsx` file and contains the component's implementation
- D) It configures TypeScript to recognize the modals folder as a module

Evaluation Criteria:

- 6/6: Exceptional — ready to apply this to any real project
- 4–5/6: Solid — revisit the one or two lessons covering missed questions
- 2–3/6: Needs review — go back through Sections 03 and 04 before moving on
- 0–1/6: Restart the course from Section 01

Section 06: Code Challenge

06.0 Structure the E-Commerce Frontend 🎯

Scenario: You have inherited a Next.js e-commerce frontend where all logic, types, API calls, and components were written by a developer who was "moving fast." Everything is in the wrong place, nothing is named consistently, and the AI agent keeps generating duplicates because it can't infer the project structure.

Requirements:

1. Design a complete target folder structure for a Next.js e-commerce app that has the following features: product listing, product detail, shopping cart, user authentication (login/register), and a user profile page
2. For each folder in your structure, write one sentence describing what lives there and what does not
3. Write a `types/index.ts` barrel file that exports the following types: `Product`, `CartItem`, `User`, `Order`
4. Write a `components/cart/index.ts` barrel file that exports: `CartSummary`, `CartItem`, `CartButton`
5. Given a function `applyDiscount(price: number, discountPercent: number): number`, decide where it lives and write the full file path including the filename
6. Given a hook `useCartTotal` that reads cart items from state and returns the formatted total price string, decide where it lives and write the full file path including the filename
7. Identify two files in the following list that are placed incorrectly and state where each one should move:
 - `/utils/fetchCart.ts` — calls `GET /api/cart/:userId`
 - `/hooks/formatPrice.ts` — pure function, no React APIs
 - `/components/ProductCard.tsx` — renders a single product
 - `/libs/authService.ts` — calls `POST /api/auth/login`

Constraints:

- Use Next.js App Router conventions (`/app` for routes)
- Use TypeScript throughout

- All shared types must live in `/types`
- All API-calling functions must live in `/services`
- All React hooks must live in `/hooks`
- No business logic inside page-level components

Expected Output / Behavior: A complete written folder structure, two barrel export files with correct TypeScript syntax, two file path decisions with justification, and two identified misplaced files with their correct destinations.